A CASE-STUDY BASED REPORT

ON

Air Pollution Anomaly detection using DBSCAN

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Praneeth Kumar Gadila 2520030257

Guntu Jothi Ganesh 2520030252

Vangapadu Sai Nikhil 2520030255

Under the Esteemed guidance of

Mr. Gaddam Ravindra Babu

Asst.Professor, Dept. of CSE

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract

Introduction .....

Background .....
Motivation for the Project .....
Overview of the Work Carried Out .....
Scope and Responsible-Use Statement .....

Problem Statement .....

Case-Study Scenario .....
Central Case-Study Question .....
Formal Problem Definition .....
Objectives that Guide the Study .....
Assumptions and Limitations .....

Dataset and Exploratory Data Analysis

About the Dataset .....
Purpose of EDA .....
Preparation Steps Actually Performed .....
EDA Exploration - Implementation .....
Summary of EDA Findings .....

Algorithm Selection and Implementation .....

Conclusion

Signature of the Team members

Abstract

Air pollution is a critical environmental issue that significantly impacts public health, ecosystems, and urban sustainability. The continuous collection of air quality data from monitoring stations generates large and complex datasets, making the manual identification of abnormal pollution events both challenging and inefficient. This project proposes an Air Pollution Anomaly Detection System that employs unsupervised machine learning techniques to identify anomalous air quality patterns without relying on labelled data.

The system utilizes key environmental parameters, including PM2.5, PM10, CO, NO2, SO2, O3, temperature, and humidity, obtained from publicly available air quality datasets. The collected data is pre-processed through data cleaning, missing valuc handling, feature selection, and feature scaling to improve model reliability and performance. Two anomaly detection algorithms, DBSCAN (Density-Based Spatial Clustering of Applications with Noise) and Isolation Forest, are implemented and comparatively analyzed. DBSCAN detects anomalies by identifying low-density observations outside meaningful clusters, while Isolation Forest isolates anomalous instances through random tree-based partitioning.

The proposed system follows the complete machine learning lifecycle, including data preprocessing, model training, evaluation, model serialization, and deployment through a lightweight web interface for real-time predictions. By enabling the early detection of unusual pollution events, the system demonstrates the practical application of anomaly detection techniques in environmental monitoring and supports timely, data-driven decision-making for smart city and public health initiatives.

1. INTRODUCTION

1.1 Background

Air quality is influenced by traffic, industrial activity, construction dust, domestic fuel use, weather, and atmospheric chemistry. Monitoring systems collect measurements such as particulate matter and gaseous pollutants. Most observations may follow recurring patterns, while a smaller number may be unusually high, unusually low, or inconsistent with other measurements. These unusual observations are called anomalies or outliers. Conventional threshold rules can be useful, but a single fixed threshold may not capture relationships among several variables. For example, an observation may be unusual because multiple pollutants rise together, or because a pollutant reading is inconsistent with the rest of the feature profile. Unsupervised anomaly detection can help surface such records for review without requiring a pre-labelled set of pollution incidents.

1.2 Motivation for the Project

• Explore whether machine-learning methods can flag unusual combinations of air-quality measurements.

• Compare a density-based method (DBSCAN) with a tree-based isolation method (Isolation Forest).

• Convert raw numeric measurements into understandable outputs such as anomaly labels, plots, and summary counts.

• Build practical understanding of the ML lifecycle, data preprocessing, scaling, clustering, and anomaly detection.

• Demonstrate how unusual observations can be reviewed by a human rather than automatically treated as confirmed incidents.

1.3 Overview of the Work Carried Out

The project workflow is organized into five practical stages:

(1) load or generate the air-pollution CSV dataset;

(2) inspect column names, data types, ranges, missing values, and duplicate rows;

(3) prepare numeric features and standardize their scales;

(4) run DBSCAN and Isolation Forest;

(5) display the resulting labels and exploratory plots. Project files previously identified include a dataset under data/air_pollution_dataset.csv, a dataset-generation script, an anomaly-detection script, a main entry point, and a dashboard. Exact implementation details should be confirmed against the current VS Code folder before a final submission.

1.4 Scope and Responsible-Use Statement

The scope is restricted to M1–M4: M1 (problem framing and ML lifecycle), M2 (data preparation and scaling), M3 (Isolation Forest), and M4 (DBSCAN and anomaly detection). The report does not claim to implement M5/M6 material, model deployment, or MLOps.

An anomaly is a statistical flag, not a diagnosis of the cause. High values may be genuine pollution events, measurement errors, calibration problems, or rare but valid conditions. The prototype should not replace calibrated sensors, official AQI calculations, expert review, or public-health guidance. If real monitoring data is used, its source, licence, collection period, units, location coverage, and known quality issues should be documented

2. PROBLEM STATEMENT

2.1 Case-Study Scenario

Consider an air-quality monitoring dataset containing measurements from different periods. Most records represent ordinary conditions, but some records may show sharp increases in PM2.5, PM10, or gases, or combinations of measurements that are rare in the dataset. Manually checking every record is time-consuming. The project investigates whether unsupervised algorithms can identify a manageable set of unusual records for further inspection.

2.2 Central Case-Study Question

Can DBSCAN and Isolation Forest identify unusual air-quality observations from multivariate pollution and weather measurements, and how do their flagged records differ?

2.3 Formal Problem Definition

Let the dataset be represented by a feature matrix X ¸ Rn×p, where n is the number of observations and p is the number of selected numeric features. Each row xn represents one observation using pollutant and weather measurements. The task is to estimate an anomaly indicator an for each row without relying on a target label. DBSCAN assigns cluster labels and marks noise observations with label -1. Isolation Forest returns a binary prediction (commonly -1 for outlier and +1 for inlier) and an anomaly score. The project compares these outputs and visualizes the records that warrant review.

2.4 Objectives that Guide the Study

• Load and inspect the air-pollution dataset and establish its structure.

• Identify missing values, duplicate records, invalid types, and implausible values where detectable.

• Prepare the numeric feature matrix and scale features so that measurement units do not dominate distance calculations.

• Apply DBSCAN to group dense patterns and identify noise points.

• Apply Isolation Forest to isolate unusual observations using randomized tree partitions.

• Compare flagged records and visualize pollutant patterns and anomaly distributions.

• Document limitations and explain why flags require contextual validation.

2.5 Assumptions and Limitations

• The selected columns are assumed to be numeric and measured in consistent units; the dataset metadata must verify this.

• Scaling is important for DBSCAN because it uses distances; feature scaling can also make the feature representation more balanced.

• If the project uses a generated/demo CSV, its patterns demonstrate program behavior but do not establish real-world pollution prevalence.

• Without trusted incident labels, precision, recall, F1-score, and classification accuracy cannot be validly calculated from the unsupervised outputs alone.

• DBSCAN is sensitive to eps and min_samples and may label many points as noise or merge nearby groups if parameters are poorly selected.

• Isolation Forest depends on settings such as contamination and random state; a flagged point is not automatically harmful or erroneous.

• The model may be affected by sensor calibration, seasonal changes, regional differences, missing context, and distribution drift

3. DATASET AND EXPLORATORY DATA ANALYSIS

3.1 About the Dataset

The project dataset is a CSV containing air-quality and weather-related measurements. The feature set used in the project plan is listed below. The exact number of rows, date range, source, and whether values are synthetic or collected from real sensors must be reported from the current CSV and its provenance; these values are intentionally not fabricated in this document.

Feature	Meaning	Why it matters
PM2.5	Fine particulate matter	Small particles; useful indicator of fine-particle pollution
PM10	Coarse/fine particulate matter up to 10 µm	Captures a broader particulate fraction
CO	Carbon monoxide	Combustion-related gas
NO2	Nitrogen dioxide	Often associated with traffic and combustion
SO2	Sulfur dioxide	Can indicate sulfur-containing fuel/industrial emissions
O2	Ozone	Secondary pollutant influenced by atmospheric chemistry
Temperature	Air temperature	Meteorological context for observed patterns
Humidity	Relative humidity or dataset-defined humidity	dity Meteorological context; confirm units in dataset documentation

3.2 Purpose of EDA

Exploratory Data Analysis (EDA) is performed before fitting the algorithms so the team can understand the dataset's structure and quality. It helps reveal missingness, unusual ranges, skewed distributions, correlations, and differences in scale. EDA also provides a baseline against which model-generated anomalies can be interpreted.

• Check dataset shape, column names, data types, and sample records.

• Calculate descriptive statistics such as count, mean, standard deviation, minimum, quartiles, and maximum.

• Count missing values and duplicate rows.

• Inspect histograms or boxplots to understand distributions and potential extreme values. • Use a correlation heatmap to inspect relationships among numeric variables. • Plot pollutant trends if a timestamp column exists and is valid.

• Visualize DBSCAN cluster/noise labels and Isolation Forest inlier/outlier labels

3.3 Preparation Steps Actually Performed / Intended Pipeline

The following sequence is the project's preparation workflow and should be matched against the current code before submission. Only steps that are present in the executed implementation should be described in the final viva as completed.

• Load: read the CSV into a pandas DataFrame and inspect its shape and columns.

• Validate: check data types, missingness, duplicates, and numeric ranges; convert malformed numeric values only when the intended meaning is clear.

• Handle missing values: use a documented strategy such as dropping rows with unusable feature values or imputing numeric values. State the exact strategy actually used.

• Remove duplicates: remove exact duplicate records only when duplicates are not meaningful repeated measurements.

• Select features: use the eight project features listed above, excluding identifiers or target labels from the unsupervised feature matrix.

• Scale: standardize the selected numeric features (for example, with StandardScaler) before DBSCAN so that variables with large numeric magnitudes do not dominate distance calculations. • Fit models: run DBSCAN and Isolation Forest on the prepared feature matrix.

• Save/visualize outputs: attach labels or scores to the observations and display summary charts.

3.4 EDA Exploration – Implementation

EDA can be implemented in Python with pandas, Matplotlib, and Seaborn (if included in the project environment). The following table specifies the outputs that should be generated and discussed. Use only charts that are actually produced by the current project.

EDA output	What to examine	How to interpret it Dataset overvie
Dataset overview	Rows, columns, dtypes, sample rows	Confirms schema and input validity
Missing/duplicate summary	Missing count per column; duplicate counts	Shows whether cleaning is required
Descriptive statistics	Mean, standard deviation, quartiles, min/max	Highlights spread and possible extreme values
Histograms/boxplots	Distribution and tails for each pollutant	Shows skew, spread, and possible extreme observations
Correlation heatmap	Pairwise correlations among numeric feature	Shows linear relationships; correlation does not imply causation
Pollutant trend plots	Pollutant values over time, if timestamps exist	Can show peaks or changing patterns
DBSCAN plot	Cluster labels versus noise label -1	Shows dense groups and points not assigned to a cluster
Isolation Forest plot	Inlier/outlier counts and score distribution	Shows how many records the fitted model flags

3.5 Summary of EDA Findings

The EDA section should report observations from the actual charts rather than expected outcomes. The following is a structured results template. Replace each bracketed item after running the current scripts.

- **Dataset size: [N rows × P columns].**
- **Missing values: [columns affected and counts]; action taken: [drop/impute/none].**
- **Duplicate records: [count] detected; [count] removed, if applicable.**
- **Feature ranges: [note any unit issues, impossible values, or highly skewed variables].**
- **Relationships: [state the strongest observed correlations from the heatmap, if generated].**

• DBSCAN output: [number of clusters excluding label -1] and [number/percentage of noise points].

• Isolation Forest output: [number/percentage flagged as anomalies] with contamination setting [value].

• Agreement between methods: [number or percentage of records flagged by both], if calculated

4. ALGORITHM SELECTION AND IMPLEMENTATION

4.1 Why These Algorithms Were Selected

DBSCAN and Isolation Forest were selected because they represent two different approaches to unsupervised anomaly detection. DBSCAN is density-based: it searches for groups of nearby observations and marks observations outside dense regions as noise. Isolation Forest is tree-based: it repeatedly partitions the feature space at random, and unusual observations tend to be isolated in fewer splits. Their complementary mechanisms make comparison useful for this project. See the scikit-learn documentation for the algorithm definitions and parameter behavior.

4.2 DBSCAN: Density-Based Spatial Clustering of Applications with Noise

Purpose: identify dense groups of similar observations and label points that do not belong to any dense group as noise.

Key parameters: eps is the neighborhood radius; min_samples is the minimum number of points needed to form a dense neighborhood. The appropriate values depend on the scaled feature space and dataset. DBSCAN does not require the number of clusters to be specified in advance.

Process: (1) receive the scaled feature matrix; (2) identify each point's eps-neighborhood; (3) identify core points meeting min_samples; (4) expand clusters from connected core points; and (5) label observations that cannot be assigned to a cluster as -1 (noise).

Interpretation: a noise point is a point that does not meet DBSCAN's density criteria. It is a candidate anomaly, not necessarily a pollution event. The algorithm can be sensitive to eps/min_samples and may struggle when normal patterns have substantially different densities.

4.3 Isolation Forest

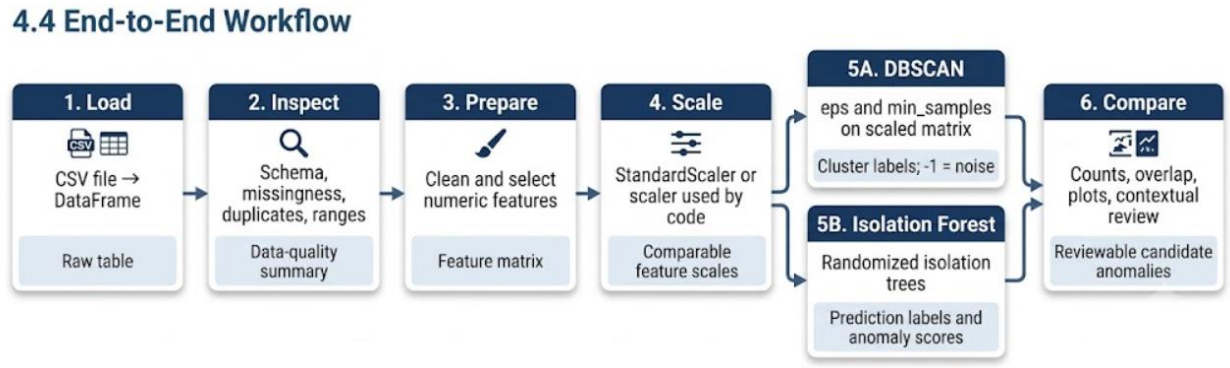
Purpose: identify observations that are easier to isolate from the rest of the dataset using random feature selection and random split values.

Process: the model builds an ensemble of randomized trees. Each tree recursively partitions the data. Observations that require shorter paths to isolate are more likely to be anomalous. The fitted model can produce predictions and anomaly scores.

Common settings: n_estimators controls the number of trees; contamination influences the threshold used to label outliers; random_state supports repeatable results. The exact settings must be copied from the implementation.

Output convention: in scikit-learn's IsolationForest, prediction -1 denotes an outlier and +1 denotes an inlier. The score convention should be read carefully: lower score_samples values indicate more abnormal observations.

4.4 End-to-End Workflow



4.5 How the Two Models Should Be Compared

• Compare the number and percentage of observations flagged by each method. • Calculate overlap: records flagged by both methods may be high-priority candidates for inspection, but agreement alone does not prove correctness.

• Inspect the pollutant values and weather context of flagged records to understand why they were unusual.

• Check sensitivity to reasonable parameter changes; large changes in anomaly counts indicate model instability.

• If verified incident labels later become available, use a labelled evaluation protocol and report precision, recall, F1-score, and confusion matrix. Do not calculate these metrics by treating one unsupervised model's predictions as ground truth for the other.

4.6 Implementation Components

The project structure previously described includes data/air_pollution_dataset.csv, src/generate_dataset.py, src/detect_anomalies.py, main.py, and dashboard.py. In the final report, explain the current implementation as follows, after verifying the filenames and behavior in VS Code:

• generate_dataset.py: creates a demonstration dataset if the project is configured to generate sample data.

• detect_anomalies.py: performs preprocessing and runs the anomaly-detection algorithms, if that is the current implementation.

• main.py: serves as the main execution entry point, if wired to the model workflow.

• dashboard.py: displays data summaries, anomaly labels, or plots, depending on the implemented dashboard.

• data/air_pollution_dataset.csv: stores the input observations used by the pipeline.

5. CONCLUSION

This project presents an unsupervised machine-learning workflow for examining air-quality measurements and identifying unusual observations. DBSCAN contributes density-based clustering and noise detection, while Isolation Forest contributes randomized tree-based isolation. Applying both methods to prepared and scaled pollutant/weather features enables the project to compare different definitions of unusual behavior and visualize records for human review.

The principal value of the prototype is its end-to-end learning workflow: dataset inspection, preprocessing, scaling, algorithm application, and result visualization. Its outputs must be interpreted cautiously. Without verified ground-truth incidents, flagged observations cannot be treated as confirmed pollution events and standard supervised classification metrics are not justified. A generated dataset is suitable for demonstrating software behavior, but real-world conclusions require documented, representative monitoring data.

Future improvements within the project's practical direction include recording dataset provenance, making parameter settings explicit, adding a repeatable results summary, and validating flagged records against trusted sensor logs or expert-reviewed events. Any extension beyond Modules M1–M4 should be included only if permitted by the course scope.